

Agent Tools and Interoperability

1. The Big Picture: Core Summary

This session from Day 2 of the Kaggle and Google five days AI agents intensive course focuses on the critical layer that makes AI agents useful in the real world: interoperability and tools. The overarching theme is the transition away from monolithic, hard-coded integrations towards open, standardized protocols that allow diverse agents to seamlessly interact with data sources, external systems, and user interfaces. By adopting standardized protocols, developers can overcome the "N-into-M" integration crisis and build scalable, distributed multi-agent architectures where agents communicate fluidly, execute dynamic UIs, and coordinate tasks securely.

2. ■ Highlights of the Conversation

- **From Monoliths to Microservices:** The shift in agent architecture mirrors web development, transitioning from bloated single-agent monoliths to distributed multi-agent networks capable of specialized tasks.
- **The N-into-M Prototyping Crisis:** Managing direct integrations between every model and every tool creates exponential technical debt, underscoring the need for open interoperability standards.
- **MCP as the USB-C for Tools:** The Model Context Protocol (MCP) standardizes tool connections, reducing complex integration webs into clean, linear scale architectures.
- **Agent-to-Agent (A2A) Coordination:** The A2A protocol serves as a universal language enabling specialized agents to discover each other via registries and delegate tasks using machine-readable cards.
- **Agent-to-UI (A2UI) Revolution:** A2UI allows agents to dynamically generate interactive, personalized interfaces on demand instead of relying on static application screens.
- **Secure Agentic Commerce:** The Universal Commerce Protocol (UCP) and Agent Payment Protocol (AP2) enable agents to build shopping carts and process payments securely while maintaining strict human-in-the-loop mandates to prevent overspending.

3. ■ Participants' Backgrounds

- **Smita Kolli:** Senior Developer Relations Engineer at Google Cloud, acting as the co-host for the intensive course and leading the interactive AMA session.
- **Anant Nawalgaria:** Senior Staff Machine Learning Engineer and Group Product Manager at Google, specializing in agentic AI, ML innovation, and serving as the co-host guiding developers through the conceptual shifts in AI software development.
- **Alan Blount:** Senior Product Manager for the Agent Platform at Google Cloud Vertex AI, instrumental in the development of A2UI and MCP protocols that define how agents render personalized user interfaces.
- **Kanchana Patlolla:** Technical Solutions Manager at Google Cloud, an expert in integrating Google Cloud databases (like Spanner) with AI workflows using secure, read-only MCP architectures.
- **Mike DeAngelo:** Developer Relations Engineer at Google, focusing on MCP integration, authentication standards, and lowering the barrier to entry for developers adopting agentic protocols in production cloud stacks.

- **Pierre:** Cloud AI Expert at Google, participating in the panel to discuss scalable and secure database architectures alongside Kanchana.

4. ■■ Questions, Answers, and Connections

Question 1: How is Google Cloud lowering the barrier for developers to integrate open protocols like MCP and A2A into production cloud stacks, and what do you see as their long-term future?

Connection to Topic: This question addresses the practical adoption of the standardized protocols discussed in the core lesson, exploring how Google Cloud provides the necessary infrastructure to implement these tools at scale.

Highlights:

- **Internet Protocol Parallels:** Comparing the foundational impact of TCP/UDP to how MCP and A2A are establishing the groundwork for future agent communications.
- **Open Standard Donations:** Ensuring protocols are open, standardized, and donated to foundations to prevent fragmented implementations.
- **Product Integrations:** Making sure developer kits, such as the Agent Development Kit (ADK), have these protocols built right in to lower the entry barrier for building agents.

Literal Transcript Answer from the Video:

Yeah. Uh, before I answer the question, you thanked everybody else, and I want to thank you, Anant and Smitha, for doing such a great job pulling this together this week. It, uh, means a lot to us at Google and the whole community, everything that you've been doing. So, in the same way that Google's putting these protocols out there, you're putting a lot of knowledge out there. So, thank you for that. Um, I think in the same way, like if you go back in time to the origins of the internet, the same way that protocols like TCP and UDP became the ingredients and the standards by which so many things run on top of, and everything since, it's very much the same way that we at Google have been trying to invest and work with the rest of the industry to come up with ways to make sure that interoperability is there for all the pieces that you need for agents to be able to work together, interact together, to be able to get to the right tools at the right time. I look at protocols like MCP and you're going to spend a whole bunch of time on that today, but just like how do I take care of a simple transaction to A2A, which is how do I actually ask another agent to take on some work on my behalf and come back and solve a problem to A2P and stateless MCP and UCP and all of the other pieces that are out there. I think the ways that we're making this easier for developers and actually like make it you know, lowering that barrier of entry is one I'm making these open standards, making sure they're donated to the right kinds of foundations that can maintain them. Um two, making sure that they are standards, making sure that it's not something that's implemented in a bunch of different ways, but that can be implemented whether you're running things here on Google or whether you're running them elsewhere. Three, we're also um integrating them into the products. So, whether you're using a ADK as agent developer kit or whether you're using one of the other software developer kits out there in the industry, you will find these same protocols implemented and used in very, very similar ways to help make sure that you can integrate those into your workflow. And then this is a vibe coding class, and so having things like agent CLI and other things, be very aware of how these different protocols work so that as you're building agents, you're able to build them right into it from scratch.

Question 2: Will protocols like A2UI replace traditional app stores with temporary highly personalized agent generated user interfaces which are rendered dynamically on demand or

where do you see this being customized for each user?

Connection to Topic: This explores the A2UI protocol's potential to disrupt traditional, static software design by using AI agents to dynamically generate interactive, context-aware interfaces for end users.

Highlights:

- **Agentic Software Future:** Moving towards an era where all software adapts to user contexts and preferences in the moment.
- **Dynamic Customization:** Interfaces will remember user preferences (e.g., buttons, sliders, or audio) and generate tailored UIs instantly.
- **Micro-Interactions:** Agents can communicate with each other to complete backend tasks (like filling out forms) and only display the UI components that require manual user confirmation.

Literal Transcript Answer from the Video:

Probably. It's hard to say. I think that we're moving to a future where all software is somewhat agentic, right? It doesn't have to everything be generated on the fly all the time, but we're going to get used to interacting with software that fits our needs and our context in the moment a little bit more closely. Um the UI layer of that, how do I interact with system, will follow a similar trajectory. It will be um personalized to me and it will remember that I like buttons or I like sliders or I prefer an audio interface or whatever. So, all of our applications and all of our user interfaces will become more personalized and also more specific to the moment. So, if my agent can talk to your agent or my subsystem can talk to your subsystem and fill in 2/3 of a form for me, I don't even need to see that form. I just have to say yes, go ahead and do it. And then I fill in the parts of the form that are necessary for this interaction. So, those seem relatively small, but they're going to end up changing how we interact with applications and going forward. Um that's not even super future-looking. Some of that is totally happening right now. Uh one of the uh anecdotes that I talk about a lot is um a tracker app on your phone uh has a UI that is provided for you by the creator of that phone that app. Um Uh but maybe you don't need all of those buttons and you want half of those buttons or you want to add something else to track or whatever. Uh it's very easy to just change the configuration of the UI. All of the same widgets persist. You just change the configuration of the UI and that static app keeps working. That's like the smallest example, but it gives you a a hint of where we think we're going with everything being configurable and customizable.

Question 3: How should database architectures evolve to natively expose secure read-only MCP servers to reduce their integration complexity?

Connection to Topic: This question tackles the security implementations necessary when exposing sensitive enterprise databases (like Spanner or SQL) to AI agents via the Model Context Protocol.

Highlights:

- **Layered Security:** Ensuring security is native to the architectural design and maintained at multiple layers.
- **Restricted Tool Registration:** Configuring MCP toolboxes to only register tools required for 'select' operations, explicitly avoiding DDL (Data Definition Language) or DML (Data Manipulation Language) access.
- **Read Replicas:** Leveraging database read replicas to give AI agents access to production data without risking the primary, write-enabled database instances.

Literal Transcript Answer from the Video:

Yeah. Oh, I'll start first. So, we talk when we talk about MCPs in general, in even in the last last iteration of the white paper, we talked about security and we talked about trust a lot. And in this paper, we did talk about why it's needed and what are the key components of it with respect to configurations and stuff. MCP is kind of becoming the standard. It is a USB-C kind of a protocol for across your databases. So, you know that. When I think about how do you make it secure, particularly with respect to, you know, avoiding a DDL, DML kind of transaction, then we need to keep it at different layers. And some of that is very much, you know, protocol-based. So, for example, if you are using, say, MCP and in Google's, you have MCP toolbox. Um and MCP toolbox, if you're using, and we talked extensively during the last iteration about it, um is how do you prevent it in such a way that it only registers tools that are needed for um a select versus giving access to a DDL or a DML. Uh and I've also seen iterations where customers use it for read replicas. Like they don't give access to the production data or they give access to the production read data, not for the right instances. Uh we need to be very specific in the architectural choices and make it um security native as part of your design of building an MCP based architecture. Uh in one MCP, which is the MCP for all of the databases, we do provide